General guidelines about writing your solutions, presentation quality, marking scheme

Coursework tasks are different from exams. They may require going beyond what we have covered explicitly in lectures, exploring alternative formulations and drawing new links between methods, to test your understanding and skills developed.

Example: in Task 2 of CW1 2024, we proposed to implement kernelised logistic regression, which required the students to combine mathematical content from kernel methods and logistic regression, and to design a new coding implementation integrating coding blocks from different notebooks.

In the coursework, you must complete the tasks according to the instructions provided in order to gain the marks.

Two other aspects of the work are needed in order to gain full marks for a given task:

- **Part of the marks are reserved and allocated to the quality of presentation, communication, and code readability.**
- **Explanation and understanding of the mathematical concepts are crucial.**

We will now discuss these two aspects in more detail.

Remark: of course, in the final, in-class part of CW2, due to time constraints, the weight of presentation quality will be much lower.

"Part of the marks are reserved and allocated to the quality of presentation, communication, and code readability."

A well-readable code and a high-quality presentation of your results are very important in general when you present your results to an external audience, or for instance in collaborative settings where you have to make your code usable by collaborators. In the coursework, it is a way to show us your understanding of the inner workings of the algorithms you implement and your correct interpretation of the results.

What does this mean in practice?

<u>In practice, we expect:</u>

1. **Comments in the code:** comment your code so that we can follow the steps you have taken and their rationale.
   Example:

```
# make predictions with the weak model
weak_predictions = predict_rss(weak_tree, X)

# update the strong model's predictions
strong_predictions -= lr * weak_predictions
```

2. **Useful outputs**: the output from your code should be clearly printed or plotted and in such a way that we can evaluate if it is correct. Summary tables are also an effective strategy to show the comparison of performance between different methods. However do not output everything - for example, it is not helpful to print the loss at every iteration over 1000 iterations.
   Example:

```
print(f"The Mean Squared Error of an ensemble of {optimalB} decision trees on the training set is:", rf_mse(optimalB_bagging_model, X_train, y_trai
print(f"The Mean Squared Error of an ensemble of {optimalB} decision trees on the test set is:", rf_mse(optimalB_bagging_model, X_test, y_test))

print(f"The R^2 score of an ensemble of {optimalB} decision trees on the training set is:", rf_r2_score(optimalB_bagging_model, X_train, y_train))
print(f"The R^2 score of an ensemble of {optimalB} decision trees on the test set is:", rf_r2_score(optimalB_bagging_model, X_test, y_test))
```

```
The Mean Squared Error of an ensemble of 20 decision trees on the training set is: 1255.0215051196553
The Mean Squared Error of an ensemble of 20 decision trees on the test set is: 3236.9719731651708
The R^2 score of an ensemble of 20 decision trees on the training set is: 0.810627680420297
The R^2 score of an ensemble of 20 decision trees on the test set is: 0.5097054051070431
```

	Metric	Model	On training set	On test set
0	R^2 score	Decision Tree	0.750364	0.474982
1	MSE	Decision Tree	1654.405151	3466.219722

   Including text in the output, rather than just printing a sequence of numbers which the reader has to interpret makes your work much easier to follow.

3. **Sensible naming of variables**: naming variables in a consistent and sensible way makes code much easier to follow, especially if you have implemented something in a slightly unconventional way. Avoid complete abbreviations of terms e.g. don't use `lct` instead of `lasso_coefficient_trajectories`.

```python
def lasso_coefficient_trajectories(lambdas, X_aug, y):
    """Computes the trajectories of the LASSO coeffiencts for a list of lambdas.
    """
    # initialise trajectories
    coeff_trajectories = []
    for lam in lambdas:
        # Run LASSO regression
        beta_lasso = minimize_ls_huber(X_aug, y, lambd=lam, n_iters=15000,
                                       step_size=1e-5,
                                       c_huber=1e-5)
        coeff_trajectories.append(beta_lasso[1:6])
    return np.vstack(coeff_trajectories)
```

   Is much better than:

```python
def l_ct(l, X, y):
    """Computes the trajectories of the LASSO coeffiencts for a list of lambdas.
    """
    # initialise trajectories
    ct = []
    for x in l:
        # Run LASSO regression
        b_lso = m_lsh(X, y, l=l, n=15000,
                                       ss=1e-5,
                                       c_h=1e-5)
        ct.append(b_lso[1:6])
    return np.vstack(ct)
```

4. **Markdown cells** providing: a brief written introduction to the task, explaining what you did and why you did it; a relevant explanation of your findings, linking them to concepts and results presented in class or in the literature. However, you do not need to simply restate the task.
Example: introduction to the task

> (1.1.1) (index)
>
> In this section I implement the decision tree. With few adaptations, I can turn the decision tree classifier from the notebook into a model for regression, i.e., to predict a continuous outcome $y \in \mathbb{R}$.
>
> There are two components in the decision tree classifier that one needs to replace for obtain a decision tree regressor:
>
> 1. The splitting step: in the decision tree classifier, we rely on an information theoretic measure (e.g., the GINI-index) as a cost function to decide the optimal column *j* and splitting value *s*. For the regressor version, one needs a loss function suited for continuous outcomes. One uses the following loss function, called the *Residual Sum of Squares* (RSS), that, for a given split into (R_1, R_2) reads:
>
> $$RSS(\boldsymbol{y}; j, s) = \sum_{\boldsymbol{x}^{(i)} \in R_1(j,s)} (y^{(i)} - \bar{y}_{R_1})^2 + \sum_{\boldsymbol{x}^{(i)} \in R_2(j,s)} (y^{(i)} - \bar{y}_{R_2})^2$$

Example: discussion of findings

> The MSE on the test set is roughly 2 times higher that of the training set ($\frac{3466}{1654} \sim 2$), relatedly the R^2 score of the test set is roughly 1.6 times lower than that of the training set ($\frac{0.75}{0.48} \sim 1.6$). This means that the decision tree fits well the training data, but does not generalise well to unseen data.
>
> It is known in general that decision trees have the tendency to overfit. Optimising some hyperparameters, like `min_samples_leaf` and `max_depth` might lead to better generalisation performance of the decision tree. Even more, switching to ensemble methods and random forests can improve it substantially, as is done in the next sections.

5. **Produce good plots documenting and summarising your findings**: see the notebook Guidelines_for_good_plots.ipynb for some guidance.

6. That the kernel is cleaned before the final run of the notebook in such a way that the notebook you upload has **cells run in order** (barring exceptional cases when the computational time of a given cell is prohibitively long). This is another way for us of checking that each cell actually produces the output printed

"Explanation and understanding of the mathematical concepts are crucial."

In the allocation of marks, we will take into account the technical correctness, logic, and clarity of explanations and mathematical interpretation of the results.

We will reward additional relevant analyses that show initiative and understanding beyond the task stated in the coursework - e.g. the thorough investigation of an aspect relevant to the observed performance of the method under consideration and its explanation.

However:

When you comment on your results, **precise pointers to your results must be provided**: generic, boiler-plate comments on a method that are not based on the specific problem and the analysis carried out by you will receive zero marks.

Similarly, the mere use or addition of calculations (or ready-made 'pipelines') that are unrelated to the task without a clear explanation and justification of your rationale will not be beneficial in itself and, in fact, can also be detrimental to the mark if it reveals lack of understanding of the required task.